

# Policy-Based Access Tokens: Privacy-Preserving Verification for Digital Identity

Kiran Pun, Daniel Gardham<sup>✉</sup> and Nick Frymann

Surrey Centre for Cyber Security, University of Surrey, United Kingdom  
<sup>✉</sup> daniel.gardham@surrey.ac.uk

**Abstract.** Passports, driving licences, and other government-issued identity documents are frequently used to prove attributes about an individual, such as their date of birth or home address. Traditional paper-based approaches are being transitioned to *digital identities*, which are becoming increasingly important for online interactions and transactions, allowing individuals to prove their identity without needing to present physical documents. However, existing solutions suffer from cumbersome primitives, for example, the European Commission is actively experimenting with Zero-Knowledge proof based solutions for the EU’s Digital Identity Wallet, or lack of functionality such as the UK’s right-to-work share codes.

In this paper, we present a new cryptographic primitive, Policy-Based Access Tokens, that allows for lightweight verification of user attributes through a service (such as a government office). We propose two variants of the scheme: PAT-I offers token unforgeability such that malicious parties cannot verify personal data without a valid token. This is then extended in PAT-II to allow for distributed delegation to a set of proxies, offering fine-grained revocation. We consider stronger security properties that prevent proxies colluding, whilst providing anonymity against the service provider. We give generic constructions of our schemes, prove their security in the standard model, and provide instantiations based on bilinear pairings. Finally, we provide a proof-of-concept implementation which demonstrates that our protocols are efficient, with token verification taking  $\approx 100\text{ms}$ .

**Keywords:** Digital Identity · Anonymous Tokens · Delegation.

## 1 Introduction

Government-issued identity documents, such as passports or driving licences, are often used to prove attributes about a person, such as name, date of birth, or home address. Traditionally, this would require presenting physical documents, but in doing so, would reveal the entire set of information contained within, often far exceeding that which is necessary. For example, a customer proving their age should not need to share their residential address when purchasing age-restricted products, as is typically the case with e.g., national identity cards.

Digital services are replacing traditional paper-based approaches. The EU’s Digital Identity (EUDI) Wallet [15] allows individuals to share their identity data online and offline for both public and private services based on the EUDI Architecture and Reference Framework (ARF). Users can collect attributes from various services and present tokens to verifiers when proving attributes—this verification step can be offline to the issuer. Whilst user control is a core principle of the design, misuse or leakage of credentials by dishonest verifiers can leak information to an unwanted third party and it does not support native time-limited verification.

Furthermore, currently used formats in the ARF such as SD-JWT (Selective Disclosure JSON Web Token) and ISO mDL (mobile Driver’s License), which rely on salted hashes, do not fully eliminate linkability concerns. Whilst using Zero-Knowledge proofs (ZKPs) to address this is under active development and discussion in the ARF, they are nonetheless known to be computationally complex and often inefficient. This complexity, coupled with the lack of standardised support in currently available secure hardware (i.e., Wallet Secure Cryptographic Devices, WSCDs), affects adoption of ZKPs in the EUDI Wallet ecosystem [23].

A lighter-weight approach is used in the United Kingdom in its Right-to-Work and Right-to-Rent share code schemes for non-British and non-Irish citizens [14, 21]. The Home Office creates a unique, nine-character alphanumeric code to enable these individuals to digitally prove their immigration status, as well as their right to work (for employers) or right to rent (for landlords). This system streamlines the verification process, replacing the need for physical documents and allows for the verification to be time-bounded, typically valid for 90 days.

This approach requires the service provider to ensure validity of the attribute at time of verification, rather than issuance, through an online check. However, the richness of this scheme is limited as it can only be used to check the types of work an individual can undertake and for how long. Similar approaches are used for the UK’s Disclosure and Barring Service (DBS) [20] to prove safety compliance. We further note the recent announcement by the UK to adopt Digital ID [34], but technical details are currently unknown.

Extending the functionality of share codes to prove additional personal information, e.g., those typically found on identity documents, would better enable digital identity. In such a setting, governments are expected to act as a high-availability, centralised, and trusted service provider. This trust and availability could be leveraged to omit the complexity challenges encountered by ZKP or anonymous credential-based approaches by providing online verification to real-time queries. However, there are two main challenges with this naïve approach.

Firstly, due to the lack of fine-grained control, a verifier could view any data field of its choosing, thus breaching the user’s privacy. This is the aforementioned drawback of a paper-based approach, where selective disclosure of data is not possible. Therefore, any solution would need to provide a user with fine-grained control over which information can be accessed, encoded in a policy. This aligns with the strong privacy controls offered by the EU digital wallet, and could also enable additional constraints such as time or role-based policies (enforced by the

service provider). Secondly, due to the semi-static nature of the share codes, and in particular, that it is not unique to the verifier, the code is identical for all queries within a refresh window. Thus, a verifying party (which we call a *proxy*) that intentionally or otherwise leaks a share code, would enable malicious parties to query and ultimately learn personal information.

Whilst shorter refresh windows limit exposure, they also increase overhead for users and the issuing service. Therefore, it is desirable that codes are bound to unique verifiers, and can be revoked at any time (by the user). However, this introduces additional privacy concerns as token redemption may leak additional information about the services with which users interact. For a centralised service such as a government ID scheme, this could leak behavioural patterns beyond what is intended by the user through analysis of meta-data. To prevent such attacks, any such scheme should also ensure anonymity of the proxies.

*Contributions.* In this paper we propose a new primitive, Policy-Based Access Tokens, that solves the challenges of extending share codes for use in wider digital identity environments. It enables users to delegate tokens that allow proxy verifiers to validate only selected information with a service.

In particular, we present two variations. PAT-I allows users to delegate tokens to proxies that are able to verify data specified in the policy. We define the *token unforgeability* security property which captures the notion that any token passing verification must have originated from an honest delegation by the user, thus preventing malicious proxies from querying the service provider without the user’s permission.

We then extend this scheme in PAT-II to enable strong user control over the tokens by distributing trust over a set of designated proxies. In particular, this scheme prevents token sharing. To allow for fine-grained proxy management, we provide the user with a revocation mechanism, utilising a verifier-local approach for improved efficiency. Furthermore, this scheme offers strong privacy guarantees that we capture in the *proxy anonymity* property which prevents service providers from linking delegated tokens.

We present generic constructions for PAT-I and PAT-II based on standard cryptographic primitives and prove that they meet token unforgeability—and proxy anonymity for the latter. We provide a proof-of-concept Python implementation and show that, even without optimisations, our protocols are very efficient, with credential delegation taking approximately 500ms and verification around 100ms for both schemes.

Finally, we observe that our Policy-Based Access Tokens scheme may find uses in other applications, which may be of independent interest. For example, when combined with secure key derivation functions, our protocol can be adopted for use in offline file/disk encryption to allow temporary access without needing to re-encrypt under new keys (a potentially computationally intensive task).

*Related work.* The United Kingdom has introduced the Digital Verification Services (DVS) register, which operates under the Digital Identity and Attributes Trust Framework (DIATF) [22]. Certified providers are authorised to perform

identity verification in regulated contexts such as tenancy applications and criminal record checks [22]. The DIATF is technology-neutral and emphasises governance, assurance levels, and compliance, whilst leaving cryptographic choices to providers. This creates space for lighter-weight cryptographic methods, such as the one proposed in this work, to improve privacy within DIATF-compliant services. In particular, it provides a privacy-preserving way to check validity and authorisation without needing to see or store raw identity data, which keeps them aligned with DIATF’s focus on user control.

Anonymous Credentials (ACs), first proposed by Chaum [9], allow users to present an attribute-based credential, such that a verifier is convinced that it has been issued to the user by a valid party—ACs could be adopted to present selective attributes to verifiers. One benefit of such an approach is that the verification step does not require the issuer to be online. However, ACs can suffer from inefficient complexity, typically due to their reliance on costly Zero-Knowledge proofs used to achieve unlinkability and anonymity (e.g., [3, 6]), although, we note other approaches rely on self-blinding techniques (as instigated by Verheul [41]).

Furthermore, credential updates require reissuing of credentials, and revocation (from the view of the issuer) is challenging. As summarised by Kakvi *et al.* [25], common methods include certificate-based and verifier-local approaches, both of which require the signer or verifier to maintain up-to-date lists, which can be cumbersome in practice. Nonetheless, a state-of-the-art scheme Server-Aided Anonymous Credentials (SAAC) by Chairattana-Apirom *et al.* [8] enables lightweight instantiations of publicly verifiable and multi-use ACs suitable compliance with existing national standards; candidate solutions [13] for the EUDI Wallet can be viewed similarly to SAAC. However, they require additional auxiliary information generated by the issuer, before verification can take place, thus adding an extra round of communication.

Chameleon hash functions [28] allow users, with knowledge of a trapdoor, to compute collisions and have been used in many credential schemes (e.g. [10, 27, 35]). Introducing policies to Chameleon hash functions have previously been explored by Tian *et al.* [39], who apply their primitive to achieve anonymity and accountability in secure blockchain rewriting, which has been extended to support revocation [42]. Note that here the *policy* refers to that of the trapdoor holder, i.e., a trapdoor owner who satisfies the access policy can amend the underlying message without affecting the hash value, rather than on verification.

Sanitizable signatures, introduced by Ateniese *et al.* [1], allow for selective disclosure of components of the message, which could be conceivably used to only share user-controlled information with a verifying party. However, the signature size is linear in the number of messages signed and thus the communication between the sanitizer and verifier is potentially orders of magnitude larger than what is required. Furthermore, there is no mechanism to enforce a verifying policy on the signature, such as a date. Revocation has been explored in the context of certificated sanitizable signatures [31], where revocation is achieved through periodic key updates. We note that policy-based sanitizable signatures have been

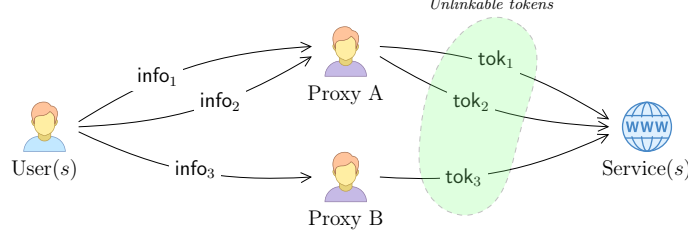


Figure 1: Our *proxy anonymity* property prevents service providers from correlating users through proxies, which could leak metadata, such as user behaviours.

proposed by Samelin and Slamanig [36], where the policy is on the sanitization rights, not verification.

Designated-verifier signatures (DVS)—not to be confused with the Digital Verification Service—first formalised by Jakobsson, Sako, and Impagliazzo [24], ensures that only a specific designated verifier can validate a signature, whilst preventing them from convincing others of its authenticity. This non-transferability is achieved by enabling the verifier to simulate valid-looking signatures themselves. Steinfeld *et al.* [38] generalised this idea with universal designated-verifier signatures (UDVS), allowing any holder of a signature to designate it to a verifier using the verifier’s public key, without the signer’s involvement. This concept has also been applied to anonymous credentials, such as in the RETRACT framework [12], where credential proofs are bound to a designated verifier to prevent misuse or data leakage. This would be unsuitable for our setting as the verifier must be fixed during the setup phase, as the token is bound to a specific verifier from the outset.

## 2 Our Policy-Based Access Tokens Scheme

In this section, we formally describe our Policy-Based Access Tokens scheme. We give the syntax and security definitions for two variations of the scheme: PAT-I and the extended PAT-II, which supports issuance to sets of proxies and their granular revocation.

An overview of the protocol for our Policy-Based Access Tokens scheme is provided in Figure 2, which uses syntax from Section 2.1. In both PAT-I and PAT-II, the user holds a shared secret with the service provider, which could be an existing account credential, share code, or identity number—for example, a driving licence number (for the DVLA in the UK), to prove the user’s identity (name) and the fact they hold a valid licence, or an insurance policy number, to prove the user has correct coverage. This shared secret is used to generate multiple tokens for proxy users, which allows verification of attributes of the user, without giving access to the secret itself. A policy  $\psi$  could specify a restricted timeframe for access (specific dates, or a single session), what data may be accessed (for example, name, home address, licence validity, account balance,

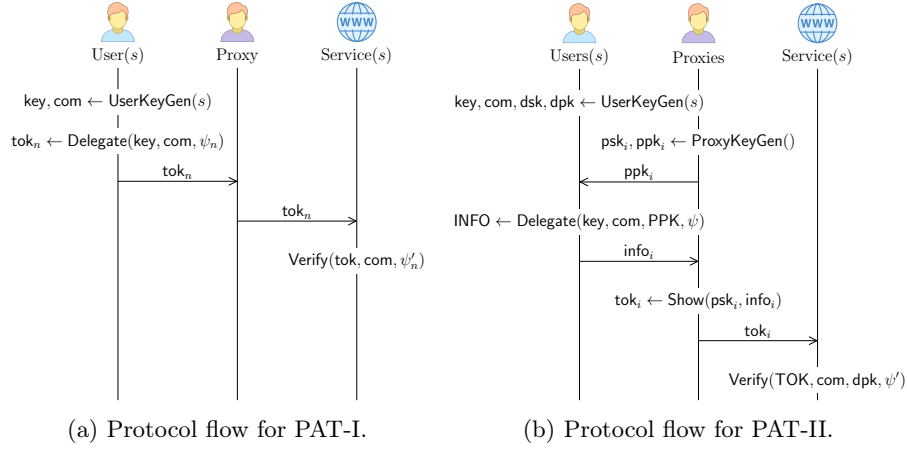


Figure 2: The Policy-Based Access Tokens protocol flows. In PAT-I, the proxy simply passes  $\text{tok}$  to the service provider to gain access. To achieve revocation and the required anonymity property, PAT-II uses a proxy key  $\text{ppk}$ , which may be reused to generate tokens many times.

etc.), or permissions the proxy has upon presenting the token (e.g., read/write, read-only).

A policy  $\psi$  is encoded in  $\text{tok}$  and is communicated to the server as part of the verification request when the proxy presents  $\text{tok}$ . The applicable  $\psi$  is therefore uniquely determined by the contents of presented  $\text{tok}$ . The server derives a context-specific interpretation  $\psi'$  by applying parameters to the current verification context. The resulting interpretation  $\psi'$  is used to determine the scope, validity period, and permissions associated with the token. For example, it may create  $\psi'$  based on  $\psi$  by inserting the current time, or an age or other attribute of the user. These context-specific parameters ultimately determine the validity of the request. In line with our design goals, policies are intentionally restricted to simple, policy-specific conditions (e.g., conjunctive statements); more expressive or complex policy languages are not considered. The exact format of the policy is application dependent, as the types of permissions encoded may vary between use cases; although some standardisation could be introduced. In PAT-II, where a user can issue tokens to sets of proxies that must collaborate during the verification phase, the policy also describes the access structure for the proxies, e.g., conjunctive, threshold, monotone. We give a simple example in Example 1.

In PAT-I, summarised in Figure 2a, the user can generate many tokens for proxy users, who need only present them directly to the service provider. The user calls  $\text{UserKeyGen}$  with the secret  $s$  to generate a key  $\text{key}$  and commitment  $\text{com}$ . This is then used as input to  $\text{Delegate}$  which may generate many tokens  $\text{tok}$ ; these are then distributed to proxies accordingly. The tokens  $\text{tok}_i$  are presented to the service provider who, using  $\text{Verify}$ , confirm its validity, which includes checks that the policy is satisfied.

To permit fine-grained user control and distributed delegation, in PAT-II (presented in Figure 2b), we allow the user to designate a particular set of proxies, which use proxy keys to generate unlinkable and revocable tokens. These can be combined in the verification step to ensure the set of proxies is valid with respect to the policy  $\psi$ , which encodes both a description of the access policy (e.g., the threshold of proxies) and other constraints e.g., date or read/write permissions.

In this scheme, the proxies generate and provide the user with a long-term proxy public key **ppk**. The user runs **Delegate** using a **ppk** to generate **info**, which includes a derived public key (which is authenticated by the proxy). The user may run **Delegate** many times to allow the proxy to access multiple service providers or to generate multiple access structures for distinct sets of proxies. The user provides **info** to the proxy, who calls **Show** with their long-term secret key **psk** to generate a token share **tok** to present to the service provider. Once a candidate set of tokens has been collected, using the **Verify** algorithm, the server confirms their validity against the policy. Proxies can be revoked at any time, with the user registering the derived public key **ppk** on a blocklist at the service provider.

The tokens provided by proxies offer *proxy anonymity* such that the service provider cannot determine the identity of the proxy, or even whether two tokens are for the same proxy. This prevents metadata leakage which could otherwise undermine users' privacy, such as disclosing user behaviours to service providers. Note that in the case where distribution of trust is not necessary, setting a trivial access structure (e.g., 1-out-of-1) recovers a simplified scheme that still benefits from proxy anonymity and revocation.

*Example 1.* A simple example of a policy is as follows: the user wants to prove they are aged 18 or above on a certain date. They would create the policy  $\psi$  as

$$\psi := (\text{over\_18} : \text{Green} \wedge \text{date} : \text{Blue})$$

The **Green** box is an assertion being made by the user, whereas the **Blue** box is a restriction on the proxy specifying when they can check this information. Together they form the policy; this should be interpreted as the user being at least 18 years of age which can be checked only on 22 June 2026. When the service provider receives this request, it computes its own context-dependent policy  $\psi'$  as

$$\psi := (\text{over\_18} : \text{Orange} \wedge \text{date} : \text{Orange})$$

Here, the **Orange** boxes are to be supplied by the service provider, enforcing validity of the asserted claims and within valid constraints. In particular, it checks the age claim against its own database and inserts  $T$  or  $F$  appropriately, and inserts the current date from a trusted time source. If either of these do not match  $\psi$ , then the scheme will reject the query, else  $\psi'$  will verify.

*Remark 1.* We observe that our construction can be used in applications wider than digital identity. In particular, we propose that it can be used to allow delegated access to encrypted files or disks. To realise this, the key commitment **com** would be used to seed a key derivation function, the key from which encrypts the

file or disk. Our PATS schemes would allow delegated recovery of the KDF seed, and thus of the encryption key. This would make our scheme PAT-I suitable for use in time-based delegation of full-disk encryption keys (e.g., LUKS<sup>1</sup>), requiring a trusted source of time (obtained from e.g., an onboard real-time clock).

Furthermore, PAT could be integrated into online authentication to permit (possibly time-bound) access to user accounts. This motivates distributed tokens as users may want to allow for secondary access but only under agreement of multiple trusted proxies.

## 2.1 Syntax for Policy-Based Access Tokens

In this section, we provide the syntax and correctness properties for both variants of the Policy-Based Access Tokens scheme. The first, PAT-I, in Definitions 1 and 2, and the second, PAT-II, in Definitions 3 and 4.

**Definition 1 (PAT-I syntax).** *A PAT-I scheme comprises four algorithms, permitting policy-based tokens to be generated from long-term shared secret,  $s$ .*

**Setup( $\lambda$ )** *This algorithm takes as input a security parameter,  $\lambda$ , generates a description of the scheme and outputs a trapdoor  $\mathbf{td}$  as part of public parameters  $\mathbf{pp}$ . Public parameters  $\mathbf{pp}$  are implicitly input to all algorithms.*

**UserKeyGen( $s$ )** *This algorithm takes as input  $s$  from the key space  $\mathcal{S}$ , or  $\perp$ , and is run by the user to generate a key  $\mathbf{key}$  and its commitment  $\mathbf{com}$ .*

**Delegate( $\mathbf{key}, \mathbf{com}, \psi$ )** *Executed by the user, for some access policy  $\psi$  from the universe of policies  $\Psi$ , and key  $\mathbf{key}$  with commitment  $\mathbf{com}$ , it generates a token  $\mathbf{tok}$ .*

**Verify( $\mathbf{tok}, \mathbf{com}, \psi'$ )** *This algorithm is run by the server. It verifies  $\mathbf{tok}$  provided by the proxy, and outputs 1 if the token is valid for the key in  $\mathbf{com}$ , and with respect to  $\psi'$ . Otherwise, it returns 0.*

**Definition 2 (PAT-I correctness).** *A PAT-I scheme is correct if, for all  $\lambda \in \mathbb{N}$ ,  $s \in \mathcal{S}$ ,  $\psi \in \Psi$ , and  $\psi' \in \Psi$ , the probability  $\Pr[\text{Verify}(\mathbf{tok}, \mathbf{com}, \psi') = 1] = 1$  given*

$$\mathbf{pp} \leftarrow \text{Setup}(\lambda); (\mathbf{key}, \mathbf{com}) \leftarrow \text{UserKeyGen}(s); \mathbf{tok} \leftarrow \text{Delegate}(\mathbf{key}, \mathbf{com}, \psi)$$

**Definition 3 (PAT-II syntax).** *A PAT-II scheme comprises seven algorithms, permitting distributed revocable policy-based tokens to be generated from long-term shared secret,  $s$ :*

**Setup( $\lambda$ )** *This algorithm takes as input a security parameter  $\lambda$ , generates a description of the scheme and outputs a trapdoor  $\mathbf{td}$  as part of public parameters  $\mathbf{pp}$ . Public parameters  $\mathbf{pp}$  are implicitly input to all algorithms.*

**UserKeyGen( $s$ )** *This algorithm takes as input  $s$  from the key space  $\mathcal{S}$ , or  $\perp$ , and is run by the user to generate a key  $\mathbf{key}$  and its commitment  $\mathbf{com}$ .*

<sup>1</sup> <https://gitlab.com/cryptsetup/LUKS2-docs>



| $\text{Exp}_{\mathcal{A}, \text{PAT-I}}^{\text{TokenUnforge}}(\lambda)$                              | $\mathcal{O}_{\text{Delegate}}(\text{key})$ on input $\psi$               |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1 : $\text{pp} \leftarrow \text{Setup}(\lambda)$                                                     | 1 : $\text{tok} \leftarrow \text{Delegate}(\text{key}, \text{com}, \psi)$ |
| 2 : $(\text{key}, \text{com}) \leftarrow \text{UserKeyGen}(\perp)$                                   | 2 : $\text{TL} \leftarrow \text{TL} \cup \{\text{tok}\}$                  |
| 3 : $(\text{tok}^*, \psi^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Delegate}}}$                  | 3 : <b>return</b> tok                                                     |
| 4 : <b>return</b> $\text{Verify}(\text{tok}, \psi, \text{com}) \wedge \text{tok}^* \notin \text{TL}$ |                                                                           |

Figure 3: Token unforgeability experiment for PAT-I.

**ProxyKeyGen()** *This algorithm is executed by the proxy. When called, it generates a proxy key pair as  $(\text{psk}, \text{ppk})$ .*

**Delegate**(key, com,  $\text{ppk}_i, \psi$ ) *Executed by the user, for some access policy  $\psi$  from the universe of policies  $\Psi$ , a set of  $n$  proxy public keys PPK, and a key key with commitment com, it returns info for each user  $i$ ,  $\text{INFO} := \{\text{info}_i\}_{i=1}^n$ .*

**Revoke**( $\text{dsk}, \text{info}_i$ ) *This algorithm is executed by the user to revoke the delegation access given to a proxy  $i$ . The user updates the revocation list RL.*

**Show**( $\text{psk}_i, \text{info}_i$ ) *Executed by a proxy  $i$ , with delegation information  $\text{info}_i$  and proxy secret key  $\text{psk}_i$ , this algorithm outputs a token  $\text{tok}_i$ .*

**Verify**(TOK, com,  $\text{dpk}, \psi'$ ) *Run by the server, it checks that each  $\text{tok} \in \text{TOK}$  is not in the revocation token list RL, it verifies if TOK is valid for the key in com, and with respect to  $\psi'$ . If these checks pass then it outputs 1, else 0.*

**Definition 4 (PAT-II correctness).** *A PAT-II scheme is correct if, for all  $\lambda \in \mathbb{N}$ ,  $s \in \mathcal{S}$ ,  $\psi \in \Psi$ ,  $\psi' \in \Psi$ , the probability  $\Pr[\text{Verify}(\text{TOK}, \text{com}, \text{dpk}', \psi') = 1] = 1$  given  $\forall i \in [1, n]$*

$\text{pp} \leftarrow \text{Setup}(\lambda)$ ;  $(\text{key}, \text{com}) \leftarrow \text{UserKeyGen}(s)$ ;  $(\text{psk}_i, \text{ppk}_i) \leftarrow \text{ProxyKeyGen}()$ ;  
 $\text{info}_i \leftarrow \text{Delegate}(\text{key}, \text{com}, \text{ppk}_i, \psi)$ ;  $\text{tok}_i \leftarrow \text{Show}(\text{psk}_i, \text{info}_i)$

## 2.2 Security of PAT-I

We define the *token unforgeability* security property for our Policy-Based Access Tokens scheme to capture a malicious proxy's ability to create tokens that successfully verify, but were not created by the user. This can be seen in the experiment defined in Figure 3; the experiment creates a set of public parameters, defines a user by running **UserKeyGen** to create challenge key  $\text{key}^*$  with commitment  $\text{com}^*$ . An adversary is then tasked to create a token  $\text{tok}^*$  that passes the verify algorithm. During this phase, it has access to a delegation oracle that creates valid tokens for  $\text{key}^*$ . To prevent the adversary from trivially winning, any tokens obtained from the oracle are not considered valid forgeries; this is enforced by the win condition on line 4 of the experiment using the list **TokList**. We note that we do not consider corruption of a server in this threat model due to the expected use case of the protocol, in which the service provider is trusted by the user. If the server were malicious or corrupt, then there would be no need to test tokens and instead allow direct adversarial access.

**Definition 5 (Token Unforgeability).** A PAT-I scheme offers Token Unforgeability if, for any PPT adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{PAT-I}}^{\text{TokenUnforge}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

The token unforgeability experiment is defined in Figure 3.

### 2.3 Security of PAT-II

We introduce an additional privacy property, *proxy anonymity*, for our PAT-II scheme that meets the requirement presented in Figure 1. It captures a fully malicious server’s ability to link tokens across proxies, i.e., token unlinkability. The experiment is formally defined in Figure 4. It creates public parameters and a user key  $\text{key}$  with commitment  $\text{com}$ . The adversary generates two sets of proxies and a policy  $\psi$ . The set of proxy keys  $\text{SL}_b$  is used to create the set  $\text{INFO}_b$  via the delegate algorithm, which is passed to the adversary.

The adversary then is required to output its guess  $b'$ , and it wins the experiment if  $b' = b$ . During the adversarial phase, we allow  $\mathcal{A}$  to interact with the scheme using oracles. It uses the oracle  $\mathcal{O}_{\text{Add}}$  to create proxies in the scheme and  $\mathcal{O}_{\text{Corrupt}}$  to corrupt them, i.e., to obtain their secret key  $\text{psk}$ . To delegate tokens to a proxy set of its choosing it calls  $\mathcal{O}_{\text{Delegate}}$  on an input  $\text{SL}$  comprising a set of public keys  $\text{ppk}_i$  for  $i \in [1, n]$ . The adversary can also use the oracle  $\mathcal{O}_{\text{Revoke}}$  to revoke  $\text{info}_i$ . There are two additional checks in the win condition of the experiment in line 9, which ensures that any of the challenge keys have not been revoked nor corrupted, as this would allow the adversary to trivially win the game. Similarly, the challenge proxy sets are of equal size and both are valid for the policy  $\psi$ , which we write as  $\psi(\text{SL}) = 1$ , otherwise  $\psi(\text{SL}) = 0$ .

**Definition 6 (Proxy Anonymity).** A PAT-II scheme offers Proxy Anonymity if, for any probabilistic polynomial time adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that

$$\left| \Pr \left[ \text{Exp}_{\mathcal{A}, \text{PAT-II}}^{\text{ProxyAnon-0}}(\lambda) = 1 \right] - \Pr \left[ \text{Exp}_{\mathcal{A}, \text{PAT-II}}^{\text{ProxyAnon-1}}(\lambda) = 1 \right] \right| \leq \nu(\lambda).$$

The proxy anonymity experiment is defined in Figure 4.

Modelled on PAT-I, *token unforgeability* for PAT-II is extended to allow for multiple proxies and a scheme that supports revocation. Precisely, it captures a malicious proxy’s ability to create tokens that successfully verify, but which were not generated by a valid set of users. As with proxy anonymity, we allow  $\mathcal{A}$  to interact with the scheme using  $\mathcal{O}_{\text{Add}}$ ,  $\mathcal{O}_{\text{Corrupt}}$ ,  $\mathcal{O}_{\text{Delegate}}$ , and  $\mathcal{O}_{\text{Revoke}}$ . The delegation oracle additionally tracks the token and public key pairs  $(\text{tok}^*, \text{ppk}^*)$  that  $\mathcal{A}$  queried. This enables the check on line 10 that ensures at least one token was not created by the delegation oracle, nor by a corrupt proxy. This also means we achieve a strong notion of unforgeability; the adversary can query the challenge proxies in  $\text{PPK}^*$ , winning provided it generates a *new* token  $\text{tok}^*$  for some policy  $\psi^*$ .

$$\text{Exp}_{\mathcal{A}, \text{PAT-II}}^{\text{ProxyAnon}-b}(\lambda)$$


---

```

1 : pp ← Setup(λ)
2 : (key, com, dsk, dpk) ← UserKeyGen()
3 : (psk0, ppk0) ← ProxyKeyGen()
4 : (psk1, ppk1) ← ProxyKeyGen()
5 : (SL0, SL1, ψ) ←  $\mathcal{A}_0^{\mathcal{O}_{\text{Delegate}}, \mathcal{O}_{\text{Revoke}}, \mathcal{O}_{\text{Add}}, \mathcal{O}_{\text{Corrupt}}}$ (key, com, dpk)
6 : b  $\xleftarrow{\$}$  {0, 1}
7 : (INFOb, SL'b) ← Delegate(key, com, SLb, ψ)
8 : TOKb ← Show(INFOb, SLb)
9 : b' ←  $\mathcal{A}_1^{\mathcal{O}_{\text{Delegate}}, \mathcal{O}_{\text{Revoke}}, \mathcal{O}_{\text{Add}}, \mathcal{O}_{\text{Corrupt}}}$ (TOKb)
10 : return b'  $\stackrel{?}{=} b \wedge (\forall \text{ppk}_i \in \{\text{SL}_0, \text{SL}_1\} : \text{ppk}_i \notin \text{RL} \cup \text{CL})$ 
       $\wedge |\text{SL}_0| = |\text{SL}_1| \wedge \psi(\text{SL}_0) = \psi(\text{SL}_1) = 1$ 

```

$$\text{Exp}_{\mathcal{A}, \text{PAT-II}}^{\text{TokenUnforge}}(\lambda)$$


---

```

1 : pp ← Setup(λ)
2 : (com, key, dsk, dpk) ← UserKeyGen()
3 : TOK*, ψ' ←  $\mathcal{A}^{\mathcal{O}_{\text{Delegate}}, \mathcal{O}_{\text{Revoke}}, \mathcal{O}_{\text{Add}}, \mathcal{O}_{\text{Corrupt}}}$ 
4 : return Verify(TOK*, com, dpk, ψ')
       $\wedge (\exists i \in [1, n] \text{ s.t. } (\text{ppk}_i^* \notin \text{CL}) \wedge (\forall \text{tok}_j \in \text{TOK}^* : (\text{tok}_j, \text{ppk}_i^*) \notin \text{TL}))$ 

```

$$\mathcal{O}_{\text{Delegate}}(\text{key}, \text{com}, \text{psk}, \text{ppk}) \text{ on input } \psi$$


---

```

1 : info ← Delegate(key, com, ppk, ψ)
2 : tok ← Show(psk, info)
3 : TL ← TL ∪ {tok, ppk}
4 : return tok

```

$$\mathcal{O}_{\text{Corrupt}} \text{ on input ppk}$$


---

```

1 : psk ← PL(ppk)
2 : CL ← CL ∪ {ppk}
3 : return psk

```

$$\mathcal{O}_{\text{Add}} \text{ when called}$$


---

```

1 : (psk, ppk) ← ProxyKeyGen
2 : PL(ppk) ← psk
3 : return ppk

```

$$\mathcal{O}_{\text{Revoke}} \text{ on input info}$$


---

```

1 : RL ← Revoke(info)
2 : return RL

```

Figure 4: Proxy anonymity and token unforgeability experiments for PAT-II. The token list, TL, inside the dotted box, is only used for token unforgeability.

**Definition 7 (Token Unforgeability).** A *PAT-II* offers Token Unforgeability if, for any probabilistic polynomial time adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{PAT-II}}^{\text{TokenUnforge}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

The token unforgeability experiment is defined in Figure 4.

### 3 Building Blocks

In this section, we formally present the syntax and security definitions of the building blocks we will use to build our Policy-Based Access Tokens scheme.

#### 3.1 Chameleon Hash Functions

Chameleon (or trapdoor) hash functions were introduced by Krawczyk and Rabin [28] as a non-standard type of collision-resistant hash function associated with a private key, also known as a trapdoor, that enables the trapdoor holder to compute collisions efficiently. The formal Chameleon hash framework used in our approach is based upon the adaptation by Camenisch *et al.* [5].

**Definition 8 (Chameleon Hash Function [5]).** A Chameleon hash function CH consists of five algorithms:

$\text{CParGen}(\lambda)$  takes as input the security parameter  $\lambda$  and outputs public parameters of the scheme. These public parameters  $pp_{ch}$  are implicitly given as input for all algorithms.

$\text{CKGen}(pp_{ch})$  generates the key pair  $(sk, pk)$  of the scheme.

$\text{CHash}(pk_{ch}, m)$  takes public key  $pk_{ch}$  and a message  $m$  to hash, returning a hash  $h$  and some randomness  $r$ .

$\text{CHashCheck}(pk_{ch}, m, r, h)$  is a deterministic algorithm that gets the public key  $pk_{ch}$ , a message  $m$ , randomness  $r$  and a hash  $h$  as the input. It returns a decision  $d \in \{0, 1\}$  indicating whether the hash  $h$  is valid.

$\text{Adapt}(sk_{ch}, m, m', r, h)$  on input of secret key  $sk_{ch}$ , the old message  $m$ , the randomness  $r$ , hash  $h$  and a new message  $m'$ , returns a new randomness  $r'$ .

*Chameleon hash correctness.* A CH scheme is correct if, for all security parameters  $\lambda \in \mathbb{N}$ , public parameters  $pp_{ch} \leftarrow \text{CParGen}(\lambda)$ , key pairs  $(sk_{ch}, pk_{ch}) \leftarrow \text{CKGen}(pp_{ch})$ , message  $m \in \mathcal{M}$ , hash and randomness  $(h, r) \leftarrow \text{CHash}(pk_{ch}, m)$ , modified message  $m' \in \mathcal{M}$ , and adapted randomness  $r' \leftarrow \text{Adapt}(sk_{ch}, m, m', r, h)$ , the following holds:

$$\text{CHashCheck}(pk_{ch}, m, r, h) = \text{CHashCheck}(pk_{ch}, m', r', h) = 1$$

| $\text{Exp}_{\mathcal{A}, \text{CHash}}^{\text{CollRes}}(\lambda)$                                            | $\mathcal{O}_{\text{Adapt}'}(\text{sk}, m, m', r, h)$                    |
|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| 1 : $\text{pp}_{\text{ch}} \leftarrow \text{CParGen}(\lambda)$                                                | 1 : <b>if</b> $\text{CHashCheck}(\text{pk}_{\text{ch}}, m, r, h) \neq 1$ |
| 2 : $(\text{sk}_{\text{ch}}, \text{pk}_{\text{ch}}) \leftarrow \text{CKGen}()$                                | 2 : <b>return</b> $\perp$                                                |
| 3 : $Q \leftarrow \emptyset$                                                                                  | 3 : $r' \leftarrow \text{Adapt}(\text{sk}_{\text{ch}}, m, m', r, h)$     |
| 4 : $(m^*, r^*, m'^*, r'^*, h^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Adapt}'}}(\text{pk}_{\text{ch}})$ | 4 : <b>if</b> $r' = \perp$ , <b>return</b> $\perp$                       |
| 5 : <b>return</b> $m'^* \notin Q \wedge m^* \neq m'^* \wedge$                                                 | 5 : $Q \leftarrow Q \cup \{m, m'\}$                                      |
| 6 : $\text{CHashCheck}(\text{pk}_{\text{ch}}, m^*, r^*, h^*) = 1 \wedge$                                      | 6 : <b>return</b> $r'$                                                   |
| 7 : $\text{CHashCheck}(\text{pk}_{\text{ch}}, m'^*, r'^*, h^*) = 1$                                           |                                                                          |

Figure 5: Collision resistance property for Chameleon hashes.

*Security properties.* We note that there are additional properties defined for this construction, such as indistinguishability and uniqueness. However, in this work, we will focus on one particular security property of Chameleon hash functions, namely collision resistance. This property ensures that, even with access to an adapt oracle, an adversary  $\mathcal{A}$  cannot find any collisions for message other than those queried to the adapt oracle.

**Definition 9 (Collision Resistance).** *CH offers Collision Resistance if, for any PPT adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that:*

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{CHash}}^{\text{CollRes}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

The collision resistance experiment is defined in Figure 5.

### 3.2 Asynchronous Remote Key Generation

ARKG [16] allows for asynchronous key delegation. It was first used in the analysis of Yubico’s backup authentication protocol for WebAuthn; it has since been used to enable anonymous delegation in the context of WebAuthn [18] and enabled strong privacy properties in group signatures [32].

**Definition 10 (Asynchronous Remote Key Generation [16]).** *An ARKG scheme consists of five algorithms, Setup, KGen, DerivePK, DeriveSK, Check, specified as follows.*

*Setup*( $\lambda$ ) *takes as input the security parameter  $\lambda$  and outputs public parameters of the scheme. We assume that  $\text{pp}_{\text{ARKG}}$  is implicit input to all other ARKG algorithms.*

*KGen*() *generates the key pair  $(\text{sk}, \text{pk})$  of the scheme when called.*

*DerivePK*( $\text{pk}, \text{aux}$ ) *takes as input the public key  $\text{pk}$  and auxiliary data  $\text{aux}$  and returns a new public key  $\text{ppk}$  and derivation data  $\text{cred}$ .*

*DeriveSK*( $\text{sk}, \text{cred}$ ) *takes input a long-term secret key  $\text{sk}$  and derivation data  $\text{cred}$  and returns derived private key  $\text{psk}$  for corresponding  $\text{ppk}$ .*

*Check*( $\text{psk}, \text{ppk}$ ) *returns 1 if  $(\text{psk}, \text{ppk})$  form a valid key pair, otherwise 0.*

| $\text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | $\mathcal{O}_{\text{pku}}(\text{sk}, \text{pk})$ when called                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 : $\text{pp} \leftarrow \text{Setup}(\lambda)$<br>2 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$<br>3 : $b \leftarrow \{0, 1\}$<br>4 : $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{pku}}}(\text{pk})$<br>5 : <b>return</b> $b' = b$                                                                                                                                                                                                                                                                                                                                                  | 1 : $(\text{pk}_0, \text{cred}) \leftarrow \text{DerivePK}(\text{pk})$<br>2 : $\text{sk}_0 \leftarrow \text{DeriveSK}(\text{sk}, \text{cred})$<br>3 : $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}()$<br>4 : <b>return</b> $(\text{sk}_b, \text{pk}_b, \text{cred})$ |
| $\text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{hwKS}}(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | $\mathcal{O}_{\text{dpk}}(\text{pk}, \text{aux})$                                                                                                                                                                                                                            |
| 1 : $\text{pp} \leftarrow \text{Setup}(\lambda)$<br>2 : $Q_{\text{pk}'}, Q_{\text{sk}'} \leftarrow \emptyset$<br>3 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\text{pp})$<br>4 : $(\text{sk}^*, \text{pk}^*, \text{cred}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{dpk}}, \mathcal{O}_{\text{dsk}}}(\text{pk})$<br>5 : $\text{sk}' \leftarrow \text{DeriveSK}(\text{sk}, \text{cred}^*)$<br>6 : <b>return</b> $\text{Check}(\text{pp}, \text{sk}^*, \text{pk}^*)$<br>7 : $\wedge \text{Check}(\text{pp}, \text{sk}', \text{pk}^*)$<br>8 : $\wedge \text{cred}^* \notin Q_{\text{sk}'}$ | 1 : $(\text{pk}', \text{cred}) \leftarrow \text{DerivePK}(\text{pk}, \text{aux})$<br>2 : $Q_{\text{pk}'} \leftarrow Q_{\text{pk}'} \cup \{(\text{pk}', \text{cred})\}$<br>3 : <b>return</b> $(Q_{\text{pk}'}, \text{pk}', \text{cred})$                                      |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | $\mathcal{O}_{\text{dsk}}(\text{sk}, \text{cred})$                                                                                                                                                                                                                           |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 1 : $\text{sk}' \leftarrow \text{DeriveSK}(\text{sk}, \text{cred})$<br>2 : $Q_{\text{sk}'} \leftarrow Q_{\text{sk}'} \cup \{\text{cred}\}$<br>3 : <b>return</b> $(Q_{\text{sk}'}, \text{sk}')$                                                                               |

Figure 6: Experiments for ARKG’s unlinkability and honest-weak key secrecy properties.

*Security properties.* An ARKG scheme provides public-key unlinkability if  $\mathcal{A}$  is not able to distinguish between derived public keys and uniformly-sampled public keys. Additionally, honest-weak private-key secrecy is provided if  $\mathcal{A}$  is not able to derive a valid key pair  $(\text{sk}^*, \text{pk}^*)$  and corresponding  $\text{cred}^*$  for an initial public key  $\text{pk}$ . Other variants of the private-key secrecy properties are provided in the original paper.

**Definition 11 (PK Unlinkability).** An ARKG scheme offers PK Unlinkability if, for any PPT adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that:

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

The public-key unlinkability experiment is defined in Figure 6.

**Definition 12 (Honest-Weak Key Secrecy).** ARKG offers Honest-Weak Key Secrecy if, for any PPT adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that:

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{hwKS}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

The honest-weak key secrecy experiment is defined in Figure 6.

| $\text{Exp}_{\mathcal{A}, \text{DS}}^{\text{EUF-CMA}}(\lambda)$                                | $\mathcal{O}_{\text{Sign}}(\text{sk}, m)$                       |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 1 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$                               | 1 : $\text{QueryList} \leftarrow \text{QueryList} \cup \{m\}$   |
| 2 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}}(\text{pk})$            | 2 : <b>return</b> $\sigma \leftarrow \text{Sign}(\text{sk}, m)$ |
| 3 : <b>if</b> $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1 \wedge m^* \notin \text{QueryList}$ |                                                                 |
| 4 : <b>then return</b> 1                                                                       |                                                                 |
| 5 : <b>else return</b> 0                                                                       |                                                                 |

Figure 7: Unforgeability (EUF-CMA) for digital signature schemes.

### 3.3 Digital Signatures

A digital signature scheme enables a user to produce a message-specific value, called a signature, which can later be verified by any third party to ensure the authenticity and integrity of the message.

**Definition 13 (Digital Signature Scheme [26]).** *A digital signature scheme consists of three algorithms  $(\text{Gen}, \text{Sign}, \text{Verify})$  such that:*

*$\text{KeyGen}(\lambda)$  outputs a pair of keys  $(\text{pk}, \text{sk})$ , where  $\text{pk}$  is the public verification key and  $\text{sk}$  is the secret signing key.*

*$\text{Sign}(\text{sk}, m)$  takes as input the secret key  $\text{sk}$  and a message  $m \in \{0, 1\}^*$ , and outputs a signature  $\sigma$ .*

*$\text{Verify}(\text{pk}, m, \sigma)$  takes as input the public key  $\text{pk}$ , a message  $m \in \{0, 1\}^*$ , and a signature  $\sigma$ , and outputs a bit  $b \in \{0, 1\}$ , where 1 indicates acceptance and 0 indicates rejection.*

*DS correctness.* For all  $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$  and  $m \in \{0, 1\}^*$ , we have:

$$\text{Verify}(\text{pk}, m, \text{Sign}(\text{sk}, m)) = 1$$

*Security properties.* In our construction, we utilise a standard digital scheme that guarantees *existential unforgeability under chosen-message attacks* (EUF-CMA). It captures the requirement that even an adversary who can obtain valid signatures on arbitrary messages of its choice should be unable to forge a signature on a new message that was not queried to the signing oracle.

**Definition 14 (EUF-CMA Unforgeability).** *A DS scheme offers the EUF-CMA Unforgeability property if, for any PPT adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that:*

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

*The EUF-CMA experiment is defined in Figure 7.*

|                                                              |                                                                     |
|--------------------------------------------------------------|---------------------------------------------------------------------|
| $\text{Exp}_{\mathcal{A}, \text{SS}}^{\text{Priv}}(\lambda)$ |                                                                     |
| 1 :                                                          | $k \leftarrow_{\$} K$                                               |
| 2 :                                                          | $(\psi, \mathcal{I}) \leftarrow \mathcal{A}_0()$                    |
| 3 :                                                          | $\{s_i\}_{i=1}^n \leftarrow \text{Share}(k, \psi)$                  |
| 4 :                                                          | $k' \leftarrow \mathcal{A}_1((i, s_i)_{i \in \mathcal{I}})$         |
| 5 :                                                          | <b>if</b> $k = k' \wedge (\mathcal{I}) \neq 1$ <b>then return</b> 1 |
| 6 :                                                          | <b>else return</b> 0                                                |

Figure 8: Privacy experiment for secret sharing schemes.

### 3.4 Secret Sharing Schemes

Secret sharing allows for a party to split a secret  $k$  into shares  $s_i$  according to some policy  $\psi$ . These shares can be combined to recover the secret  $k$  if and only if the set of shares  $\mathcal{I}$  satisfy  $\psi$ , which we denote as  $\psi(\mathcal{I}) = 1$ . In the literature, the set  $\mathcal{I}$  is called an authorised set. In our work, we allow  $\psi$  to be any monotone access structure, which includes threshold and conjunctive policies, and which we consider to be most appropriate for our work.

**Definition 15 (Secret Sharing Schemes [29]).** *A Secret Sharing Scheme SS consists of two algorithms:*

**Share**( $k, \psi$ ) *On input of a secret  $k$  and policy  $\psi$ , this algorithm outputs shares  $s_1, \dots, s_d$ .*  
**Recover**( $\{(i, s_i)\}_{i=1}^t, \psi$ ) *on input of the pairs  $(i, s_i)$  for all  $i \in [1, t]$  and candidate policy  $\psi$ , it either outputs  $k$  or  $\perp$ .*

*SS correctness.* An SS scheme is correct if, for every qualified set  $\mathcal{I}$ , reconstruct recovers the original secret  $k$ . That is, for any  $\psi$ , and any  $\mathcal{I}$  such that  $\psi(\mathcal{I}) = 1$ , we have the following with all but negligible probability:

$$\{s_i\}_{i=1}^n \leftarrow \text{Share}(k, \psi) \implies k \leftarrow \text{Recover}(\{(i, s_i)\}_{i=1}^t, \psi)$$

*Security properties.* Security properties of these schemes require that  $k$  cannot be obtained by any corrupt set of shares that do not satisfy  $\psi$ .

**Definition 16 (Privacy).** *An SS scheme offers Privacy if, for any PPT adversary  $\mathcal{A}$ , there exists a negligible function  $\nu(\cdot)$  such that:*

$$\Pr \left[ \text{Exp}_{\mathcal{A}, \text{SS}}^{\text{Priv}}(\lambda) = 1 \right] \leq \nu(\lambda).$$

*The privacy experiment is defined in Figure 8.*



|                                                                                           |                                                                                      |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Setup</b> ( $\lambda$ )                                                                | <b>Delegate</b> (key, com, $\psi$ )                                                  |
| 1 : $\text{pp}_{\text{ch}} \leftarrow \text{CH.ParGen}(\lambda)$                          | 1 : <b>parse</b> key = ( $s, r$ ) and com = $h$                                      |
| 2 : $\text{td}, \text{pk}_{\text{ch}} \leftarrow \text{CH.CKGen}()$                       | 2 : $r' \leftarrow \text{CH.Adapt}(\text{pp.td}, \psi, s, r, h)$                     |
| 3 : <b>return</b> $\text{pp} = (\text{pp}_{\text{ch}}, \text{td}, \text{pk}_{\text{ch}})$ | 3 : <b>return</b> tok $\leftarrow r'$                                                |
| <b>UserKeyGen</b> ( $s$ )                                                                 | <b>Verify</b> (tok, com, $\psi'$ )                                                   |
| 1 : <b>if</b> $s = \perp$ , <b>then</b> $s \leftarrow \{0, 1\}^\lambda$                   | 1 : <b>parse</b> com = $h$                                                           |
| 2 : $(h, r) \leftarrow \text{CH.Hash}(\text{pp.pk}_{\text{ch}}, s)$                       | 2 : <b>if</b> $\text{CH.CHashCheck}(\text{pp.pk}_{\text{ch}}, \psi', \text{tok}, h)$ |
| 3 : <b>return</b> key = ( $s, r$ ), com = $h$                                             | 3 : <b>then return</b> 1, <b>else return</b> 0                                       |

Figure 9: Algorithms for our PAT-I construction.

## 4 Generic Construction of Policy-Based Access Tokens

We now present the constructions for PAT-I and PAT-II, based on the building blocks introduced in Section 3, and prove their security properties in this section.

### 4.1 PAT-I Construction

Here we detail the construction of PAT-I which is based on simplistic use of a Chameleon hash functions to achieve secure token delegation. To begin, the server executes the **Setup** algorithm to generate public parameters  $\text{pp}$  by running Chameleon hash **CParGen** algorithm followed by **CKGen** to generate the public key  $\text{pk}_{\text{CHash}}$  and trapdoor  $\text{td}$  associated to a Chameleon hash function.

A user runs the **UserKeyGen** algorithm with input of their long-term key  $s$ . Using  $\text{pk}_{\text{CHash}}$ , it creates a hash  $h$  and randomness  $r$  by executing **CHash**. To delegate to a proxy, the user then runs the **Delegate** algorithm. This consists of executing **CHash.Adapt** with trapdoor  $\text{td}$ , policy  $\psi$ , hash  $h$ , long term key  $s$ , and randomness  $r$  as the input to produce delegation token  $\text{tok} = r'$ . Intuitively, it creates a collision on hash  $h$  for the new message  $\psi$ .

The token is verified by the server by running the **Verify** algorithm. This entails running **CH.CHashCheck** on input of the user's hash  $h$  and the proxy's token  $\text{tok} = r'$ . The server provides its instance of the policy  $\psi'$  from the policy description. If this check passes, and **CH.CHashCheck** passes, then the server outputs 1 and accepts the token, otherwise it returns 0. Intuitively, to pass verify, the proxy must have obtained a valid token, i.e., produced a collision on  $h$  for a message  $\psi'$  from the user. Otherwise, it would have broken collision resistance of the Chameleon hash **CHash**. The algorithms are presented in Figure 9.

*Correctness.* The correctness of the PAT-I construction presented in Figure 9 follows from the correctness of the building blocks **DS** and **CHash**.

**Theorem 1.** *PAT-I satisfies Token Unforgeability if the Chameleon hash function **CH** offers Collision Resistance.*

*Proof.*  $\text{Game}_0$  is defined to be the experiment  $\text{Exp}_{\mathcal{A}, \text{PAT-I}}^{\text{TokenUnforge}}(\lambda)$ .

$\text{Game}_1$  is defined as  $\text{Game}_0$  but with the following changes: we introduce an additional check in line 5 of the game, namely, we check that  $k \neq \psi^*$ . We observe that the difference in the advantage of the adversary winning  $\text{Game}_0$  and  $\text{Game}_1$  is bound by the probability that the adversary guesses  $s$ . Since  $s \in \{0, 1\}^\lambda$ , the probability that this event occurs is  $2^{-\lambda}$ , and thus the difference in advantage for  $\mathcal{A}$  between these two games is negligible.

We now examine the advantage an adversary  $\mathcal{A}$  has against  $\text{Game}_1$ . In particular, we claim that any adversary that wins  $\text{Game}_1$  can be used to build an adversary  $\mathcal{B}$  against the collision resistance property of CH. The adversary  $\mathcal{B}$  is going to invoke its experiment and receives some challenge parameters  $\text{pp}_{ch}$  and key  $\text{sk}_{\text{CHash}}$ . It passes public parameters to the token unforgeability adversary  $\mathcal{A}$  and sets the trapdoor to be  $\text{td} = \text{sk}_{\text{CHash}}$ . Adversary  $\mathcal{B}$  must also simulate the delegate oracle to  $\mathcal{A}$ , however, it can run the delegate algorithm honestly as it has knowledge of  $(\text{td}, s, r, h)$  and  $\mathcal{A}$  provides the input  $\psi$ .

After some time, the adversary  $\mathcal{A}$  returns the tuple  $(\text{tok}^*, \psi^*)$ . Then  $\mathcal{B}$  builds its response to the CR game by constructing the tuple  $(h, m = k, r, m' = \psi^*, r' = \text{tok}^*)$ . Then we observe that if  $\mathcal{A}$  wins its token unforgeability game, then  $\mathcal{B}$  wins its collision resistance game.

Thus, the sequence of games  $\text{Game}_0$  and  $\text{Game}_1$  we have shown that:

$$\text{Adv}_{\mathcal{A}, \text{PAT-I}}^{\text{TokenUnforge}}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{CHash}}^{\text{CollRes}}(\lambda).$$

Hence we conclude that PAT-I satisfies *Token Unforgeability* if the Chameleon hash function CH offers *Collision Resistance*.  $\square$

## 4.2 PAT-II Construction

This section details the construction of PAT-II, which is constructed from a Chameleon hash function CH, a digital signature DS, an Asynchronous Remote Key Generation scheme ARKG, and a secret sharing scheme SS.

As before, the server creates the public parameters  $\text{pp}$  of the scheme by running the Setup algorithm. In particular, it executes  $\text{CHash.CParGen}$ , followed by the CKGen algorithm to generate the public key  $\text{pk}_{\text{CHash}}$  and trapdoor  $\text{td}$  associated to a Chameleon hash function, then calls the setup for ARKG and DS. A user runs the UserKeyGen algorithm with their long term key  $s$  as input. Using  $\text{pk}_{\text{CHash}}$ , it creates a hash  $h$  and randomness  $r$  by executing CHash. Furthermore, it creates a revocation key pair  $(\text{dsk}, \text{dpk}) \leftarrow \text{DS.KGen}()$ .

In PAT-II, proxies also need key pairs  $(\text{psk}, \text{ppk})$ . This is generated in the ProxyKeyGen algorithm by running ARKG.KGen. In order to delegate to a (set of) proxy(ies), the user then runs the Delegate algorithm. It creates a derived public key  $\text{ppk}'$  on the proxy's public key  $\text{ppk}$  to produce delegation credentials  $\text{cred}$ . Then, as before, this comprises executing CHash.Adapt with trapdoor  $\text{td}$ , policy  $\psi$ , hash  $h$ , long term key  $s$ , and randomness  $r$  as the input to produce delegation token  $\text{tok} = r'$ . In this step, the policy  $\psi$  includes the proxy's derived public key  $\text{ppk}'$  to bind the collision randomness  $r'$  to the proxy. Then,  $r'$  is split

|                                                                                      |                                                                                          |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Setup</b> ( $\lambda$ )                                                           | <b>Delegate</b> (key, com, PPK, $\psi$ )                                                 |
| 1 : $\text{pp}_{\text{ch}} \leftarrow \text{CH.ParGen}(\lambda)$                     | 1 : <b>parse</b> key = ( $s, r$ ) and com = $h$                                          |
| 2 : $\text{td}, \text{pk}_{\text{ch}} \leftarrow \text{CH.CKGen}()$                  | 2 : $(\text{PPK}', \text{CRED}) \leftarrow \text{ARKG.DerivePK}(\text{PPK})$             |
| 3 : <b>return</b> pp = ( $\text{pp}_{\text{ch}}, \text{td}, \text{pk}_{\text{ch}}$ ) | 3 : $r' \leftarrow \text{CH.Adapt}(\text{pp.td}, \psi, h, s, r)$                         |
| <b>UserKeyGen</b> ( $s$ )                                                            | 4 : $\{r'_i\} \leftarrow \text{SS.Share}(r', \psi)$                                      |
| 1 : <b>if</b> $s = \perp$ , <b>then</b> $s \leftarrow \{0, 1\}^\lambda$              | 5 : <b>return</b> info $_i = (r'_i, \text{cred}_i, \text{ppk}'_i) \forall i \in [1 : n]$ |
| 2 : $(h, r) \leftarrow \text{CH.Hash}(\text{pp.pk}_{\text{ch}}, s)$                  | <b>Show</b> (psk $_i$ , info $_i$ )                                                      |
| 3 : $(\text{dsk}, \text{dpk}) \leftarrow \text{DS.KeyGen}()$                         | 1 : <b>parse</b> info $_i = (r'_i, \text{cred}_i, \text{ppk}'_i)$                        |
| 4 : <b>return</b> key = ( $s, r$ ), com = $h$ ,<br>dsk, dpk                          | 2 : $\text{psk}' \leftarrow \text{ARKG.DeriveSK}(\text{psk}, \text{cred})$               |
| <b>ProxyKeyGen</b> (pp)                                                              | 3 : <b>if</b> psk $'_i = \perp$ <b>then abort</b>                                        |
| 1 : $(\text{psk}, \text{ppk}) \leftarrow \text{ARKG.KeyGen}()$                       | 4 : $\pi_i \leftarrow \text{DS.Sign}(\text{psk}'_i, r'_i)$                               |
| 2 : <b>return</b> psk, ppk                                                           | 5 : <b>return</b> tok = ( $r'_i, \pi_i, \text{ppk}'_i$ )                                 |
| <b>Revoke</b> (dsk, info)                                                            | <b>Verify</b> (TOK, com, dpk, $\psi'$ )                                                  |
| 1 : <b>parse</b> ppk' from info                                                      | 1 : <b>parse</b> tok $_i = (r'_i, \pi_i, \text{ppk}'_i)$ and com = $h$                   |
| 2 : $\text{RL} \leftarrow \text{RL} \cup \{\text{ppk}'\}$                            | 2 : $r' \leftarrow \text{SS.Recover}(\text{TOK}, \psi)$                                  |
| 3 : $\sigma_{\text{RL}} \leftarrow \text{Sign}(\text{dsk}, \text{RL})$               | 3 : <b>if</b> CH.CHashCheck( $\text{pp.pk}_{\text{ch}}, \psi', r', h$ )                  |
| 4 : <b>return</b> RL, $\sigma_{\text{RL}}$                                           | 4 : $\wedge \text{ppk}' \notin \text{RL}$ for each $i \in [1, n]$                        |
|                                                                                      | 5 : $\wedge \text{DS.Verify}(\pi_i, \text{ppk}'_i)$ for each $i \in [1, n]$              |
|                                                                                      | 6 : $\wedge \text{DS.Verify}(\sigma_{\text{RL}}, \text{dpk})$                            |
|                                                                                      | 7 : <b>then return</b> 1, <b>else return</b> 0                                           |

Figure 10: Algorithms for our PAT-II construction.

by the secret sharing scheme SS into  $n$  shares  $r'_i$  according to the policy  $\psi$ . To revoke a proxy, the user appends  $\text{ppk}'$  to the revoke list RL and signs it using revocation key dsk.

A proxy processes its token tok using the Show algorithm to randomise their public key, yet demonstrates that it is the intended recipient of the token. To do this, it signs the tok with its derived secret key  $\text{psk}'$  (corresponding to  $\text{ppk}'$ ) computed from the DeriveSK algorithm, to create the signature  $\pi$ . Intuitively, this randomisation is what enables proxy anonymity.

A set of proxies present a set of tokens TOK (containing keyset PPK) to be verified by the server, which runs the Verify algorithm. It reconstructs  $r'$  from the proxies' tokens  $\text{tok}_i = r'_i$ , by running the Recover algorithm of the SS scheme. The verifier ensures that, on input of the user's hash  $h$ , CH.CHashCheck verifies against the server-provided policy  $\psi'$ . Furthermore, it validates the authenticity of the revocation list RL (using dpk) and also checks that  $\text{ppk}' \notin \text{RL}$  for each  $\text{ppk}'$  in PPK, and validates each token by verifying the signature  $\pi_i$ . If all checks pass, then the server outputs 1 and accepts the token, otherwise it returns 0.

Intuitively, the proxy must have obtained a valid token, otherwise it would have broken collision resistance of the Chameleon hash CHash, broken privacy of the secret sharing scheme SS, forged a signature  $\pi$  for a derived public key of a different proxy, or been able to recover  $\text{psk}'$  from a  $\text{ppk}'$ , breaking honest-weak security of ARKG. A formal description of the algorithms is presented in Figure 10.

*Remark 2.* We note PAT-II can support fine-grained delegation control through revocation of a single proxy by setting the proxy set to be size 1 and omitting the secret sharing step. However, we include it in our scheme for more generality.

*Correctness.* The correctness of the PAT-II construction follows from the correctness of the building blocks DS, CHash, SS, and ARKG.

**Theorem 2.** *A PAT-II scheme satisfies Proxy Anonymity if ARKG offers PK Unlinkability.*

*Proof.* Let  $\text{Game}_0^{(b)}$  be defined as  $\text{Exp}_{\mathcal{A}}^{\text{ProxyAnon-}b}(\lambda)$ , indexed on  $b$ .

Let  $\text{Game}_1^{(b)}$  be defined as  $\text{Game}_0^{(0)}$ , i.e., we fix the challenge bit  $b = 0$ . We argue that the advantage of  $\mathcal{A}$  distinguishing between  $\text{Game}_1^{(b)}$  and  $\text{Game}_0^{(b)}$  is bound by an adversary  $\mathcal{B}$  against the unlinkability property of ARKG.

To see this, consider line 6 in both games, and notice that the only input in the Delegate algorithm affected by the choice of  $b$  is  $\text{ppk}_b$ . The function CH.Adapt in Delegate, which provides  $\text{tok}_b$ , uses the derived public key  $\text{ppk}'_b$ , derived from public key  $\text{ppk}_b$  via  $\text{ARKG.DerivePK}(\text{ppk}_b, \text{aux})$ . If  $\text{ppk}'_b$  is unlinkable from  $\text{ppk}_b$ ,  $\text{tok}_b$  is also unlinkable from  $\text{ppk}_b$ . Note the secret sharing scheme SS only depends on key, which is fixed; thus does not impact the adversary's ability to guess  $b$ .

We introduce an adversary  $\mathcal{B}$  against the unlinkability property of ARKG.  $\mathcal{B}$  invokes its experiment and creates the challenge for  $\mathcal{A}$  with  $\text{pk}_0 = \text{ppk}_b$  and the fixed inputs  $(\text{td}, \psi, \text{ppk}, s, r, h)$ . After some time, the adversary  $\mathcal{A}$  outputs  $(\text{tok}_b, \text{cred}_b, \text{ppk}'_b)$ .  $\mathcal{A}$  outputs its guess at the challenge bit  $b$ , which  $\mathcal{B}$  forwards as its response in the unlinkability experiment.  $\mathcal{B}$  wins if  $\mathcal{A}$  also wins. Thus:

$$\left| \Pr[\text{Game}_1^{(1)} = 1] - \Pr[\text{Game}_0^{(1)} = 1] \right| \leq \text{Adv}_{\mathcal{B}, \text{ARKG}}^{\text{pku}}(\lambda)$$

We observe that the advantage of the adversary distinguishing  $\text{Game}_1^{(0)}$  and  $\text{Game}_1^{(1)}$  is necessarily  $\frac{1}{2}$  as it is independent of the bit  $b$ . Similarly,  $\text{Game}_0^{(0)}$  and  $\text{Game}_1^{(0)}$  are also indistinguishable as there is no change. Finally, we have shown that any adversary that can distinguish  $\text{Game}_0^{(1)}$  and  $\text{Game}_1^{(1)}$  can be used to build an adversary  $\mathcal{B}$  against the unlinkability property of ARKG. Thus, by the triangle inequality, we conclude:

$$\text{Adv}_{\mathcal{A}, \text{PAT-II}}^{\text{ProxyAnon}}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{ARKG}}^{\text{pku}}(\lambda)$$

Thus PAT-II satisfies *Proxy Anonymity* if ARKG offers *PK Unlinkability*.  $\square$

**Theorem 3.** *A PAT-II scheme satisfies Token Unforgeability if ARKG offers Honest-Weak Key Secrecy, CHash offers Collision Resistance, DS offers EUF-CMA Unforgeability and SS offers Privacy.*

*Proof.* We observe that the win condition requires that a token provided by the adversary passes verification and was not obtained through the delegation oracle. Passing verification requires 4 checks: (1)  $\text{CH.CHashCheck}(\text{td}, \psi', \text{ppk}', r', h)$ , (2)  $\text{DS.Verify}(\pi, \text{ppk}')$ , (3)  $\text{DS.Verify}(\sigma_{RL}, \text{dpk})$ , and (4)  $\text{ppk}' \notin \text{RL}$ . The adversary  $\mathcal{A}$  could attack any of the checks 1–3 in order to create a forged token, whereas check 4 prevents a trivial win.

Denote  $E_i$  as the adversary winning the experiment by attack check (i). We now consider the reduction for  $E_1$ , and split further into two subgames conditioned on whether the shares contained in TOK successfully recover tok and were created by the game. Thus, let  $E_1^a$  be the case that  $\text{tok}^* = \text{tok}$ , and let  $E_1^b$  be the case  $\text{tok}^* \neq \text{tok}$ .

In  $E_1^a$ , the adversary returns a set of shares that create the delegated collision tok. However, the game prevents the adversary from corrupting all proxies; there must be at least one honest proxy. To set up this reduction, the adversary  $\mathcal{B}$  invokes its experiment against privacy of the secret sharing scheme. It challenges  $\mathcal{A}$  against  $E_1^a$  and waits for it to output a policy  $\psi$ , which may contain many additional constraints, but will also describe an access structure for the secret sharing policy, denoted  $\tilde{\psi}$ . The adversary  $\mathcal{B}$  forwards  $\tilde{\psi}$  to the privacy game. Next, it is issued with a set of shares  $\{i, s_i\}_{i \in \mathcal{J}}$  for some maximal non-accepting set  $\mathcal{J}$ . When  $\mathcal{A}$  calls the delegate oracle on a proxy belonging to  $\mathcal{J}$ , these shares are embedded as  $\text{tok}_i^*$ . Eventually,  $\mathcal{A}$  outputs a set of tokens TOK.  $\mathcal{B}$  runs the Recover algorithm to produce a candidate secret  $k'$ , which it forwards to the privacy experiment. It wins the experiment if  $k = k'$ . Since  $\mathcal{J}$  is non-accepting, any time  $\mathcal{A}$  wins  $E_1^a$ ,  $\mathcal{B}$  wins against privacy of the secret sharing scheme.

We observe the experiment  $E_1^b$  follows the argument of Theorem 1, so we omit this for brevity. The conclusion is that any adversary able to win game  $E_1^b$  can be used to create an adversary  $\mathcal{C}$  against collision resistance of CHash, since it provided tokens that produced a distinct but valid collision. Thus, we conclude:

$$\Pr[E_1 = 1] \leq \text{Adv}_{\mathcal{B}, \text{SS}}^{\text{Priv}}(\lambda) + \text{Adv}_{\mathcal{C}, \text{CHash}}^{\text{CollRes}}(\lambda)$$

We now consider  $E_2$ .  $\text{Game}_0$  is defined to be the experiment  $\text{Exp}_{\mathcal{A}, \text{PAT-II}}^{\text{TokenUnforge}}$  (conditioned on the event  $E_2$ ).  $\text{Game}_1$  is defined as  $\text{Game}_0$  but with the following change: we change the requirement in line 4 of the game from the adversary providing  $\text{psk}^*$  instead of  $\psi^*$ . We observe that the difference in the advantage of the adversary winning  $\text{Game}_0$  and  $\text{Game}_1$  is bound by the probability that the adversary is able to create a signature  $\pi^*$  without knowledge of the secret key  $\text{psk}^*$ , i.e., the EUF-CMA property.  $\mathcal{B}$  embeds the challenge key  $\text{pk}^*$  as  $\text{ppk}^*$ , and it sets  $\text{psk}^* = \perp$ . We note that  $\mathcal{B}$  can simulate the delegate oracle for the target  $\text{ppk}^*$  by forwarding queries to its signing oracle. If  $\mathcal{A}$  wins, then  $\{\text{tok}^*, \text{ppk}^*\} \notin \text{TL}$ , which means that it did not query the delegate oracle for the token. Thus, any

signature  $\pi^*$  is a valid forgery and wins the EUF-CMA game. Therefore, we have:

$$\Pr[\text{Game}_0 - \text{Game}_1] \leq \text{Adv}_{\mathcal{D}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

We claim that any adversary that wins  $\text{Game}_1$  can be used to be build an adversary  $\mathcal{B}$  against the honest-weak key secrecy property of ARKG. Adversary  $\mathcal{B}$  invokes its experiment and receives challenge parameters  $\text{sk}_{\text{psk}}$  and key  $\text{pk}_{\text{ppk}}$ . It passes public parameters to the token unforgeability adversary  $\mathcal{A}$  and sets the proxy's public key to be  $\text{pk} = \text{ppk}$ . Adversary  $\mathcal{B}$  must also simulate the delegate oracle to  $\mathcal{A}$ , for which it uses  $\mathcal{O}_{\text{ppk}^*}$ . After some time, the adversary  $\mathcal{A}$  returns the tuple  $(\text{tok}^*, \text{psk})$ . Then  $\mathcal{E}$  builds its response to the key secrecy game by deriving  $\text{psk}'$  using the  $\text{psk}$  provided by  $\mathcal{A}$ . We observe that if  $\mathcal{A}$  provides a correct  $\text{psk}'$ , then  $\mathcal{E}$  wins its key secrecy game. Thus:

$$\Pr[\text{E}_2 = 1] \leq \text{Adv}_{\mathcal{E}, \text{ARKG}}^{\text{hwKS}}(\lambda) + \text{Adv}_{\mathcal{D}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

We now consider  $\text{E}_3$ . In particular, the adversary can attack the signature verification in line 1 of the verify algorithm, which verifies RL.  $\text{Game}_0$  is defined to be the experiment  $\text{Exp}_{\mathcal{A}}^{\text{TokenUnforge}}(\lambda)$  (conditioned on the event  $\text{E}_3$ ).

$\text{Game}_1$  is defined as  $\text{Game}_0$ , except we change the requirement in line 4 of the game from the adversary providing  $\sigma_{RL}$  to  $\text{dsk}$ . We observe that the difference in the advantage of the adversary winning  $\text{Game}_0$  and  $\text{Game}_1$  is bound by the probability that the adversary is able to create a signature  $\sigma_{RL}$  without knowledge of the secret key  $\text{dsk}$ . This probability is bound by the EUF-CMA property of the digital signature. In particular,  $\mathcal{D}$  embeds the challenge key as  $\text{dpk}$ , and it sets  $\text{dsk} = \perp$ . We note that  $\mathcal{D}$  can simulate the revoke oracle for the target  $\text{dpk}$  by forwarding queries to its signing oracle. If  $\mathcal{A}$  wins, it is able to create a forgery  $\sigma_{RL}$ . Thus  $\mathcal{D}$  wins the EUF-CMA game. Thus, we have

$$\Pr[\text{E}_3 = 1] \leq \text{Adv}_{\mathcal{D}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

which gives a total bound of

$$\text{Adv}_{\mathcal{A}, \text{PAT-II}}^{\text{TokenUnforge}}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{SS}}^{\text{Priv}}(\lambda) + \text{Adv}_{\mathcal{C}, \text{CHash}}^{\text{CollRes}}(\lambda) + 2\text{Adv}_{\mathcal{D}, \text{DS}}^{\text{euf-cma}}(\lambda) + \text{Adv}_{\mathcal{E}, \text{ARKG}}^{\text{hwKS}}(\lambda).$$

Hence, we conclude PAT-II satisfies *Token Unforgeability* if ARKG offers *Honest-Weak Key Secrecy*, CHash offers *Collision Resistance*, DS offers *EUF-CMA Unforgeability* and SS offers *Privacy*.  $\square$

## 5 Instantiation and Implementation

Our Policy-Based Access Tokens constructions are built from generic primitives, so we state here suitable primitives from which one can instantiate our scheme. For efficiency reasons, we opt for a group-based instantiation. For the Chameleon hash we use the implementation by Tian [40], for the DS scheme we use ECDSA [33] as a standardised primitive, and for ARKG, we use the generalised construction from Frymann *et al.* [16], which is compatible with pairing

Table 1: Timing and sizes, averaged over 100 runs on an Intel i5 CPU (2.7GHz), of our Python implementation. For PAT-II, we use one proxy with a trivial policy.

| Protocol | Sizes (kB) |           |       | Timing (ms) |      |        |
|----------|------------|-----------|-------|-------------|------|--------|
|          | User key   | Proxy key | Token | Delegate    | Show | Verify |
| PAT-I    | 6.96       | –         | 6.88  | 408         | –    | 85.3   |
| PAT-II   | 7.04       | 1.64      | 8.59  | 593         | 58.9 | 113    |

groups [19]. User secret keys comprise two elements in  $2\lambda\text{bits}$  and,  $2\lambda\text{bits} + \mathbb{Z}_p^*$  (for  $p = \Theta(2^\lambda)$ ), respectively. The user public key in PAT-II is in  $\mathbb{G}$  (cyclic group of order  $n$ ). We have `tok`, which has  $\lambda\text{bits}$  and  $2\lambda\text{bits} + 2\mathbb{Z}_n^* + \mathbb{G}$ , for PAT-I and PAT-II respectively. The size of `tok` is determined solely by the Chameleon hash parameters (i.e., the randomness space  $\mathcal{R}$ ) and the parameters of the signature scheme, and is therefore independent of the size of the policy  $\psi$ . Hence, a more complex policy  $\psi$  will not increase the size of `tok`. In PAT-II, proxy keys (`psk`, `ppk`) belong to  $\mathbb{Z}_n \times \mathbb{G}$ . If we compare this to approaches based on anonymous credentials, which are much more costly, e.g., the state-of-the-art SAAC scheme by Chairattana-Apirom *et al.* [8] has a credential size of  $3\mathbb{G} + 9\mathbb{Z}_n$  (for pairings) and  $7\mathbb{G} + 15\mathbb{Z}_n$  (for DDH). SAAC also requires input from a trusted helper before the showing phase, only achieving security guarantees in the random oracle model.

By using verifier local revocation, our delegation time is constant in the number of delegations and revocations, but verification is linear in the size of the revocation list `RL` and with the number of proxies ( $N$ ), i.e., it has order  $\mathcal{O}(N \cdot |\text{RL}|)$ . However, since the revocation list `RL` is typically orders of magnitude larger than the number of proxies ( $N$ ),  $\mathcal{O}(|\text{RL}|)$  is the dominant factor determining practical performance. Asymptotically, this matches many state-of-the-art anonymous credential schemes such as that of Connolly *et al.* [11], however, some schemes (e.g. [8]) could support logarithmic set membership proofs to achieve  $\mathcal{O}(\sqrt{|\text{RL}|})$  [2]. Table 1 details concrete performance metrics.

## 6 Conclusion

We introduced a new primitive, Policy-Based Access Tokens, enabling privacy-preserving digital ID schemes, comprising two variants. PAT-I offers token unforgeability against an adversary that can query tokens and PAT-II, which extends this to capture multiple proxies and introduces a revocation mechanism. We define a new property, *proxy anonymity*, capturing token unlinkability which affords the user privacy against the verifying server, thus preventing metadata surveillance. We gave generic constructions and proved they meet our security properties under standard assumptions in the standard model. Our schemes were instantiated in the group setting using a proof-of-concept implementation, writ-

ten in Python. Benchmarking demonstrates efficiency, with delegation taking 408 and 593ms, and verification 85.3 and 113ms, for PAT-I and PAT-II respectively.

*Future Work.* Given a recent focus from standardisation bodies to develop quantum-resistant cryptography, it would be desirable to consider post-quantum instantiations for Policy-Based Access Tokens as future work. We note that lattice-based Chameleon hash functions (e.g. [7, 30]) and ARKG [4, 17, 37] exist, therefore such an instantiation is feasible.

## References

1. Ateniese, G., Chou, D.H., de Medeiros, B., Tsudik, G.: Sanitizable Signatures. In: Computer Security – ESORICS 2005, pp. 159–177. Springer Berlin Heidelberg (2005)
2. Benarroch, D., Campanelli, M., Fiore, D., Gurkan, K., Kolonelos, D.: Zero-knowledge proofs for set membership: efficient, succinct, modular. *Des. Codes Cryptography* **91**(11), 3457–3525 (2023). <https://doi.org/10.1007/s10623-023-01245-1>
3. Bootle, J., Lyubashevsky, V., Nguyen, N.K., Sorniotti, A.: A Framework for Practical Anonymous Credentials from Lattices. In: CRYPTO 2023. Springer Nature Switzerland (2023)
4. Brendel, J., Clermont, S., Fischlin, M.: Post-quantum Asynchronous Remote Key Generation for FIDO2. In: ASIACRYPT 2024, pp. 465–493. Springer-Verlag (2024). [https://doi.org/10.1007/978-981-96-0891-1\\_15](https://doi.org/10.1007/978-981-96-0891-1_15)
5. Camenisch, J., Derler, D., Krenn, S., Pöhls, H.C., Samelin, K., Slamanig, D.: Chameleon Hashes with Ephemeral Trapdoors. In: Public-Key Cryptography – PKC 2017. Springer Berlin Heidelberg (2017)
6. Camenisch, J., Lysyanskaya, A.: An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation. In: EUROCRYPT 2001. Springer Berlin Heidelberg (2001)
7. Cash, D., Hofheinz, D., Kiltz, E., Peikert, C.: Bonsai Trees, or How to Delegate a Lattice Basis. In: EUROCRYPT 2010. Springer (2010)
8. Chairattana-Apirom, R., Harding, F., Lysyanskaya, A., Tessaro, S.: Server-Aided Anonymous Credentials. In: Advances in Cryptology – CRYPTO 2025, pp. 291–324. Springer Nature Switzerland (2025)
9. Chaum, D.: Security without identification: transaction systems to make big brother obsolete. *Commun. ACM* **28**(10), 1030–1044 (1985). <https://doi.org/10.1145/4372.4373>
10. Choi, J., Jung, S.: A handover authentication using credentials based on chameleon hashing. *Communications Letters, IEEE* **14** (2010)
11. Connolly, A., Deschamps, J., Lafourcade, P., Perez Kempner, O.: Protego: Efficient, Revocable and Auditable Anonymous Credentials with Applications to Hyperledger Fabric. In: Progress in Cryptology – INDOCRYPT 2022: 23rd International Conference on Cryptology in India, Kolkata, India, December 11–14, 2022, Proceedings. Springer-Verlag (2022). [https://doi.org/10.1007/978-3-031-22912-1\\_11](https://doi.org/10.1007/978-3-031-22912-1_11)
12. Debes, H.B., Giannetsos, T.: RETRACT: Expressive Designated Verifier Anonymous Credentials. In: Proceedings of the 18th International Conference on Availability, Reliability and Security. ARES ’23. ACM (2023)



13. Desmoulins, N., Dumanois, A., Kane, S., Traoré, J.: Making BBS Anonymous Credentials eIDAS 2.0 Compliant, *Cryptology ePrint Archive*, Paper 2025/619 (2025). <https://eprint.iacr.org/2025/619>.
14. Disclosure and Barring Service, Check a tenant’s right to rent in England: Use their share code, (2024). <https://www.gov.uk/view-right-to-rent>.
15. European Commission, European Digital Identity, <https://commission.europa.eu/strategy-and-policy/priorities-2019-2024/europe-fit-digital-age/european-digital-identity>.
16. Frymann, N., Gardham, D., Kiefer, F., Lundberg, E., Manulis, M., Nilsson, D.: Asynchronous Remote Key Generation: An Analysis of Yubico’s Proposal for W3C WebAuthn. In: *ACM CCS 2020*, pp. 939–954. ACM (2020)
17. Frymann, N., Gardham, D., Manulis, M.: Asynchronous Remote Key Generation for Post-Quantum Cryptosystems from Lattices. In: *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023*. IEEE (2023)
18. Frymann, N., Gardham, D., Manulis, M.: Unlinkable Delegation of WebAuthn Credentials. In: *Computer Security – ESORICS 2022*. Springer Nature Switzerland
19. Frymann, N., Gardham, D., Manulis, M., Nartz, H.: Generalised Asynchronous Remote Key Generation for Pairing-Based Cryptosystems. In: *Applied Cryptography and Network Security, ACNS 2023*. LNCS, Springer, Heidelberg (2023)
20. Government Digital Service, Disclosure & Barring Service, (2024). <https://www.gov.uk/government/organisations/disclosure-and-barring-service>.
21. Government Digital Service, Prove your right to work to an employer. Tech. rep., (2024)
22. Government Digital Service, UK digital identity and attributes trust framework: Gamma 0.4 (pre-release). In: *Department for Science, Innovation and Technology* (2025). <https://www.gov.uk/government/publications/uk-digital-identity-and-attributes-trust-framework-04/uk-digital-identity-and-attributes-trust-framework-gamma-04-pre-release>
23. Group, E.D.I.C.: Architecture and Reference Framework (ARF), (2025). <https://eudi.dev/latest/architecture-and-reference-framework-main/%5C#7435-risks-and-mitigation-measures-related-to-user-privacy>
24. Jakobsson, M., Sako, K., Impagliazzo, R.: Designated Verifier Proofs and Their Applications. In: *EUROCRYPT*. Springer Berlin Heidelberg (1996)
25. Kakvi, S.A., Martin, K.M., Putman, C., Quaglia, E.A.: SoK: Anonymous Credentials. In: *Security Standardisation Research*. Springer Nature Switzerland (2023)
26. Katz, J.: “Digital Signatures”. In: *Digital Signatures*. Springer US, 2010. Chap. 1, pp. 9–10.
27. Kirupanithi, D.N., Antonidoss, A.: Self-Sovereign Identity Management System on blockchain based applications using Chameleon Hash. In: *2nd International Conference on Smart Electronics and Communication (ICOSEC)* (2021)
28. Krawczyk, H., Rabin, T.: Chameleon Signatures. In: *Proceedings of the Network and Distributed System Security Symposium, NDSS 2000*, San Diego, California, USA. The Internet Society (2000)
29. Krenn, S., Lorünser, T.: An Introduction to Secret Sharing: A Systematic Overview and Guide for Protocol Selection (2023)
30. Li, Y., Liu, S.: Tagged Chameleon Hash from Lattices and Application to Redactable Blockchain. In: *Public-Key Cryptography – PKC 2024*. Springer Nature Switzerland (2024)
31. Lin, H.-Y., Tsai, T.-T., Wu, H.-L.: A Revocable Certificateless Sanitizable Signature Scheme With Batch Verification. *IEEE Access* **12**, 143392–143400 (2024)

32. London, C., Gardham, D., Dragan, C.C.: Dynamic Group Signatures with Verifier-Local Revocation. In: 2025 IEEE 38th Computer Security Foundations Symposium (CSF), pp. 473–488. IEEE Computer Society (2025). <https://doi.org/10.1109/CSF64896.2025.00026>. <https://doi.ieeecomputersociety.org/10.1109/CSF64896.2025.00026>
33. National Institute of Standards and Technology, Digital Signature Standard (DSS). Tech. rep., (2013). <https://doi.org/10.6028/nist.fips.186-4>
34. Prime Minister’s Office, New Digital ID scheme to be rolled out across UK, (2025). <https://www.gov.uk/government/news/new-digital-id-scheme-to-be-rolled-out-across-uk>.
35. Sabramooz, M.R., Zhang, K.: ChameleonRev: A Novel Approach to Efficient and Granular Control Credential Revocation on the Blockchain. In: 2024 6th International Conference on Blockchain Computing and Applications (BCCA) (2024)
36. Samelin, K., Slamanig, D.: Policy-Based Sanitizable Signatures. In: CT-RSA 2020, pp. 538–563. Springer International Publishing (2020)
37. Stebila, D., Wilson, S.: Quantum-Safe Account Recovery for WebAuthn. In: 19th Asia Conference on Computer and Communications Security, ASIA CCS. ACM (2024)
38. Steinfeld, R., Bull, L., Wang, H., Pieprzyk, J.: Universal Designated-Verifier Signatures. In: ASIACRYPT 2003. Springer Berlin Heidelberg (2003)
39. Tian, Y., Li, N., Li, Y., Szalachowski, P., Zhou, J.: Policy-based Chameleon Hash for Blockchain Rewriting with Black-box Accountability. In: 36th Annual Computer Security Applications Conference. ACSAC ’20. ACM (2020)
40. Tian, Y.: Accountable Blockchain Rewriting. GitHub repository <https://github.com/Yangguang-Tian-Surrey/Accountable-Blockchain-Rewriting> (2020)
41. Verheul, E.R.: Self-Blindable Credential Certificates from the Weil Pairing. In: ASIACRYPT 2001. Springer Berlin Heidelberg (2001)
42. Xu, S., Ning, J., Ma, J., Xu, G., Yuan, J., Deng, R.H.: Revocable Policy-Based Chameleon Hash. In: ESORICS. Springer International Publishing (2021)